Asignatura MATI · Curso 2026–27

Guía de uso del trabajo

El problema del viajante sobre la red de carreteras de España

Repositorio, notebook*, web interactiva, algoritmos, estudio y memoria*



Raimundo, Pablo, Juan y Jiahao

Noviembre de 2026

Cómo leer esta guía

Esta guía es para el equipo: explica cómo usar el trabajo, sobre todo la **web interactiva**, y cómo seguir colaborando en el repositorio. No hace falta leerla entera. Según lo que quieras hacer:

Quiero…	Ve a…
Abrir la web y probar los algoritmos	capítulo 3
Preparar la presentación	sección 3.9
Programar o cambiar un algoritmo	capítulo 4
Entender qué hay en el repositorio	capítulo 1
Ejecutar el *notebook* o repetir el estudio	capítulo 5
Compilar la memoria o la guía	capítulo 6
Algo no funciona	capítulo 7

Índice general

Cómo leer esta guía II

1. El trabajo de un vistazo 2
- 1.1. Qué es . . . 2
- 1.2. Los ocho algoritmos y quién los hizo . . . 2
- 1.3. Qué hay en el repositorio . . . 3

2. Puesta en marcha 4
- 2.1. Requisitos . . . 4
- 2.2. Obtener el repositorio . . . 4
- 2.3. Abrir el *notebook* . . . 4

3. La web interactiva 5
- 3.1. Cómo abrirla . . . 5
- 3.2. Recorrido por la pantalla . . . 5
- 3.3. Elegir las ciudades . . . 6
- 3.4. Elegir la versión del grafo . . . 7
- 3.5. Controlar la ejecución . . . 7
- 3.6. Qué se ve con cada algoritmo . . . 7
- 3.7. Límites: por qué a veces no se puede ejecutar . . . 9
- 3.8. La pestaña Carrera . . . 10
- 3.9. Guion sugerido para la presentación . . . 11
- 3.10. Publicarla en internet (GitHub Pages) . . . 11

4. Trabajar con los algoritmos 12
- 4.1. Cómo funciona cada notebook de algoritmo . . . 12
- 4.2. Pasos para añadir o cambiar un algoritmo . . . 13
- 4.3. Reglas . . . 13
- 4.4. Integrar los algoritmos (quien coordina) . . . 14
- 4.5. Pedir una versión del grafo . . . 14
- 4.6. Qué escribir en tu *notebook* . . . 15

5. *Notebook*, estudio y datos 16
- 5.1. Las cinco partes del *notebook* . . . 16
- 5.2. Repetir el estudio empírico . . . 16
- 5.3. Regenerar el grafo grande . . . 17

6. Compilar la memoria y esta guía 18
- 6.1. Obtener Tectonic . . . 18
- 6.2. Compilar . . . 18
- 6.3. Regenerar las figuras de la memoria . . . 18

6.4. Convenciones tipográficas 19

7. Problemas frecuentes **20**

7.1. Glosario 21

Capítulo 1

El trabajo de un vistazo

1.1 Qué es

El trabajo estudia el *problema del viajante* sobre la red de carreteras real de España: dado un conjunto de ciudades, encontrar la ruta más corta que pasa por todas y vuelve al principio. Se construyó un grafo con las carreteras (OpenStreetMap), se calcularon las distancias entre 75 ciudades y se implementaron y compararon ocho algoritmos. El resultado son tres entregables:

- el ***notebook*** `trabajo_tsp.ipynb` (el trabajo, en cinco partes);
- la **memoria** (`memoria/memoria.pdf`), que lo cuenta todo; y
- la **web interactiva** (`web/`), que anima los algoritmos y sirve de apoyo en la presentación.

1.2 Los ocho algoritmos y quién los hizo

Tabla 1.1. Algoritmos del trabajo.

Algoritmo	Autor	Tipo	Para qué sirve
Fuerza bruta	Pablo	exacto	Probar todas las rutas: solo con muy pocas ciudades.
Held–Karp	Raimundo	exacto	Programación dinámica: óptimo hasta unas 20 ciudades.
DFJ	Jiahao	exacto	Programación entera: óptimo con 75 ciudades en unos 4 s.
Vecino más cercano	Pablo	heurístico	Ir siempre a la ciudad más cercana: rápido, pero mediocre.
2-opt	Raimundo	heurístico	Mejorar una ruta quitando cruces: el mejor equilibrio.
Christofides	Raimundo	aproximado	Garantiza no pasarse más de un 50 % del óptimo (grafo no dirigido).
Recocido simulado	Juan	metaheurística	Acepta empeoramientos que van disminuyendo, como al forjar metal.
Genético	Raimundo	metaheurística	Una población de rutas que evoluciona.

1.3 Qué hay en el repositorio

Tabla 1.2. Carpetas y archivos principales.

Ruta	Qué contiene
`trabajo_tsp.ipynb`	El *notebook* principal (Partes 1 a 5).
`parte3/`	Un *notebook* por algoritmo, la plantilla, `fichas.json` y `herramientas.py`.
`anexos/`	Anexo A (descargar el mapa), B (construir el grafo) y C (ejecutar el estudio).
`data/ciudades.csv`	Las 75 ciudades.
`data/versiones/`	16 tablas de distancias y el catálogo de las 480 versiones.
`data/resultados/`	Resultados del estudio (archivos CSV).
`web/`	La web interactiva.
`memoria/`	La memoria en LaTeX y su PDF.
`guia/`	Esta guía.
`README.md`, `REGISTRO.md`, `CONTRIBUTING.md`	Documentación, registro de autoría y uso de IA, y guía para colaborar.

Ojo. El grafo grande (`data/espana_red_maestra_secundarias.graphml`, 108 MB) **no está en git**. No lo necesitas para usar la web, programar algoritmos ni repetir el estudio: se regenera con los anexos A y B (unos 11 minutos) solo si hace falta.

Capítulo 2

Puesta en marcha

2.1 Requisitos

Necesitas **Python 3.10 o superior** y **git**. Para la web, además, un navegador moderno (Chrome, Edge o Firefox). No hace falta instalar nada más para usarla.

2.2 Obtener el repositorio

```
git clone https://github.com/raigarper/MATI_TRABAJO.git
cd MATI_TRABAJO
pip install -r requirements.txt
```

Si ya lo tenías clonado, trae lo último con `git pull origin main`. Si trabajas en tu rama (`dev-pablo`, `dev_jiahao`, `dev_juan`…), primero ponte en ella con `git checkout <tu-rama>` y luego haz el `pull`.

Consejo. Con `pip install -r requirements.txt` se instala también `pulp[cbc]`, que necesita DFJ. Si algo falla al importar una librería, es lo primero que hay que revisar.

2.3 Abrir el *notebook*

```
jupyter lab trabajo_tsp.ipynb
```

El *notebook* se ejecuta de arriba abajo (*Kernel* → *Restart & Run All*) y tarda unos 15–25 minutos, sobre todo por las pruebas de los algoritmos. La Parte 1 carga el grafo grande; si no lo tienes, **salta la Parte 1 y ejecuta desde la Parte 2**: las tablas de distancias ya están calculadas.

Ojo. Si el equipo tiene poca memoria libre, cierra el navegador y otros programas antes de ejecutar el *notebook* completo: la Parte 1 necesita varios gigabytes y el sistema puede cortar la ejecución.

Capítulo 3

La web interactiva

3.1 Cómo abrirla

La web necesita un servidor local (los navegadores no permiten abrir `index.html` haciendo doble clic):

En Windows.
Doble clic en `web/iniciar_web.bat`. Arranca el servidor y abre el navegador solo. Para pararlo, cierra la ventana negra.

En cualquier sistema.
Desde la carpeta `web/`:

```
python -m http.server 8000
```

y abre `http://localhost:8000` en el navegador.

No necesita internet: todo lo que usa (incluidas las librerías de DFJ y Christofides) está dentro de la carpeta. También se puede abrir desde su dirección pública (apartado 3.10).

3.2 Recorrido por la pantalla

La figura 3.1 muestra el Laboratorio. Tiene cuatro zonas:

1. **Panel izquierdo (ciudades):** qué ciudades se usan y con qué versión del grafo.
2. **Barra de algoritmos (arriba):** los ocho, cada uno con su color.
3. **Mapa (centro):** la ruta se va construyendo sobre las carreteras reales.
4. **Panel derecho:** explicación del algoritmo, pseudocódigo con la línea actual resaltada, qué está pasando, los datos (coste, gráfica) y la ficha.

Debajo del mapa están los **controles de reproducción**.

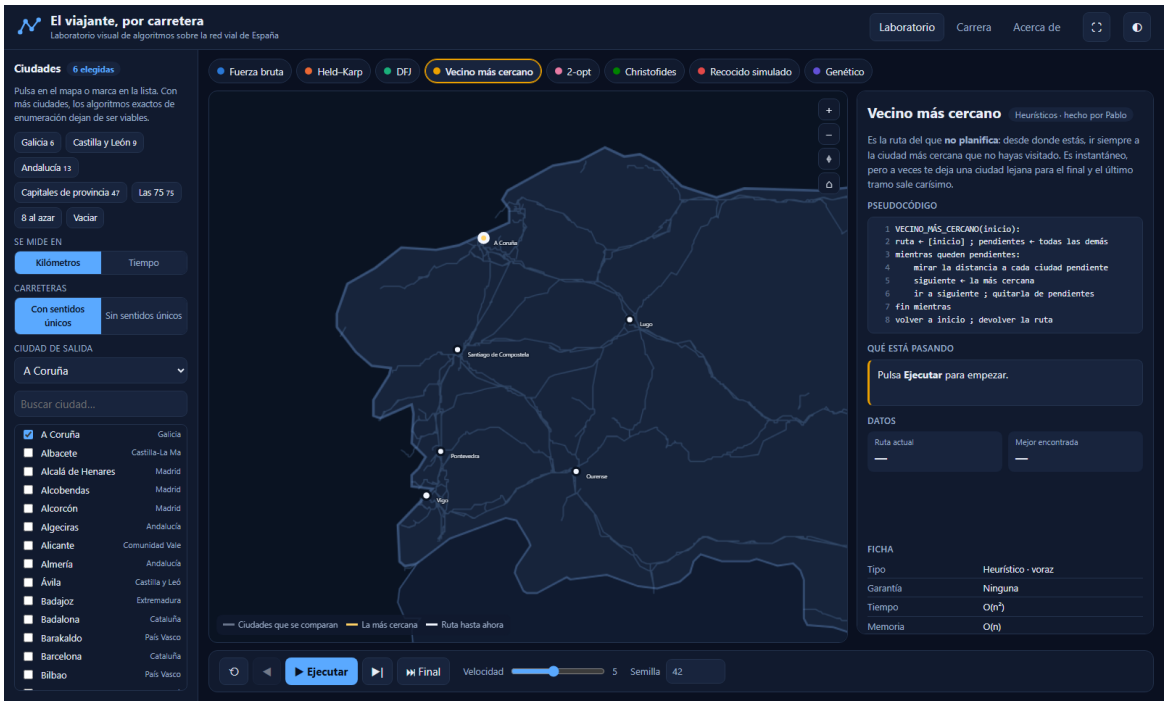


Figura 3.1. Pantalla inicial del Laboratorio, con las 6 ciudades de Galicia y el vecino más cercano seleccionado.

3.3 Elegir las ciudades

- **Ejemplos predefinidos** (botones de arriba del panel): *Galicia* (6), *Castilla y León* (9), *Andalucía* (13), *Capitales de provincia* (47), *Las 75*, *8 al azar* y *Vaciar*. Al elegir un ejemplo, el mapa se acerca a las ciudades.
- **Pulsar sobre el mapa:** una ciudad gris se añade; una ciudad blanca se quita. También se puede marcar y desmarcar en la lista de la izquierda (con un buscador).
- **Ciudad de salida:** la ciudad desde la que empieza la ruta (por defecto, Madrid si está elegida).

3.4 Elegir la versión del grafo

- **Se mide en: Kilómetros o Tiempo** (horas de coche). La ruta más corta no es siempre la más rápida.
- **Carreteras: Con sentidos únicos o Sin sentidos únicos.** Con sentidos únicos, ir de A a B puede costar distinto que de B a A (como en la realidad); sin ellos, ida y vuelta cuestan lo mismo.

3.5 Controlar la ejecución

Tabla 3.1. Controles del reproductor.

Botón	Qué hace	Atajo
Ejecutar / Pausar	Empieza o detiene la animación (se convierte en *Continuar* y, al acabar, en *Repetir*).	`Espacio`
Paso (▸|)	Avanza un solo paso.	→
Atrás (◂)	Retrocede un paso.	←
Final	Salta directamente al resultado (en la fuerza bruta puede tardar unos segundos).	
Reiniciar ((flecha circular))	Vuelve al principio.	
Velocidad	Deslizador de 1 (lento, para explicar) a 10 (muy rápido).	
Semilla	Para el genético y el recocido: con la misma semilla repiten el mismo resultado.	

Además, la **rueda del ratón** hace zoom en el mapa y **arrastrar** lo mueve. Los botones de la esquina del mapa son + y − (zoom), ⊕ (encuadrar las ciudades elegidas) y △ (ver toda la península). Arriba a la derecha hay un botón de **pantalla completa** (para presentar) y otro para cambiar entre **tema oscuro y claro**.

3.6 Qué se ve con cada algoritmo

Tabla 3.2. Qué muestra el mapa en cada algoritmo.

Algoritmo	Qué se ve
Fuerza bruta	Cada ruta que prueba, y un contador de rutas probadas sobre las posibles. Marca cuándo encuentra una mejor.
Held–Karp	Los mejores caminos parciales por tamaño de grupo de ciudades y, al final, la ruta reconstruida hacia atrás.
DFJ	La solución del problema fácil partida en **circuitos sueltos** (cada uno de un color); luego las reglas que se añaden y la solución final.
Vecino más cercano	Las ciudades que compara (línea discontinua), la elegida (amarillo) y la ruta hasta ahora.
2-opt	Los dos tramos que se quitan (rojo discontinuo) y los dos que se ponen (verde).
Christofides	El árbol mínimo (azul), el emparejamiento (rosa) y, al final, la ruta sin repetir ciudades.
Recocido simulado	La mejor ruta, la actual (tenue) y los tramos que se cambian; la temperatura baja con los niveles.
Genético	La mejor ruta y 8 rutas de la población (tenues); la gráfica muestra cómo baja el coste por generación.

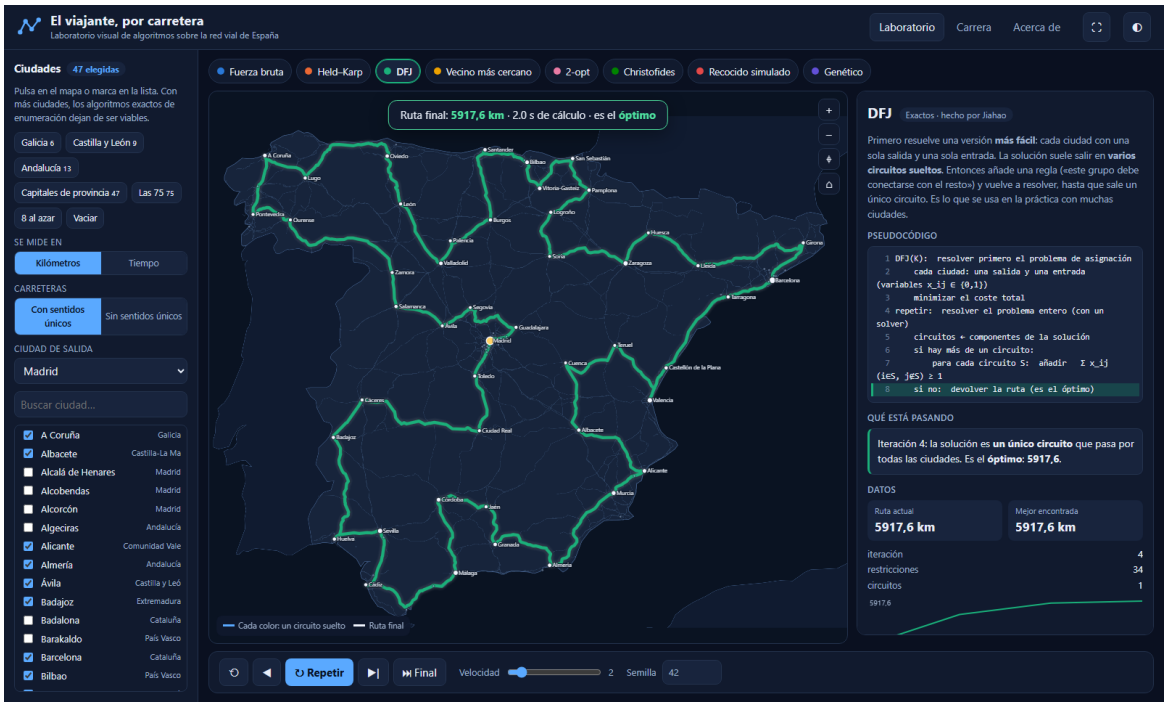


Figura 3.2. Resultado final de DFJ con las 47 capitales: 5.917,6 km, el óptimo. Arriba, el cuadro con el coste y el tiempo de cálculo.

3.7 Límites: por qué a veces no se puede ejecutar

Algunos algoritmos no se pueden aplicar a cualquier conjunto de ciudades, y la web lo explica con un aviso amarillo (figura 3.3). Es parte del resultado del estudio:

- **Fuerza bruta:** hasta 10 ciudades en la animación. Con 47 ciudades habría 10^{58} rutas.
- **Held–Karp:** hasta 18 ciudades (su memoria se duplica con cada ciudad).
- **Christofides:** solo con *Sin sentidos únicos*; el aviso incluye un botón para cambiarlo.
- Mínimo **3 ciudades** para cualquier algoritmo.

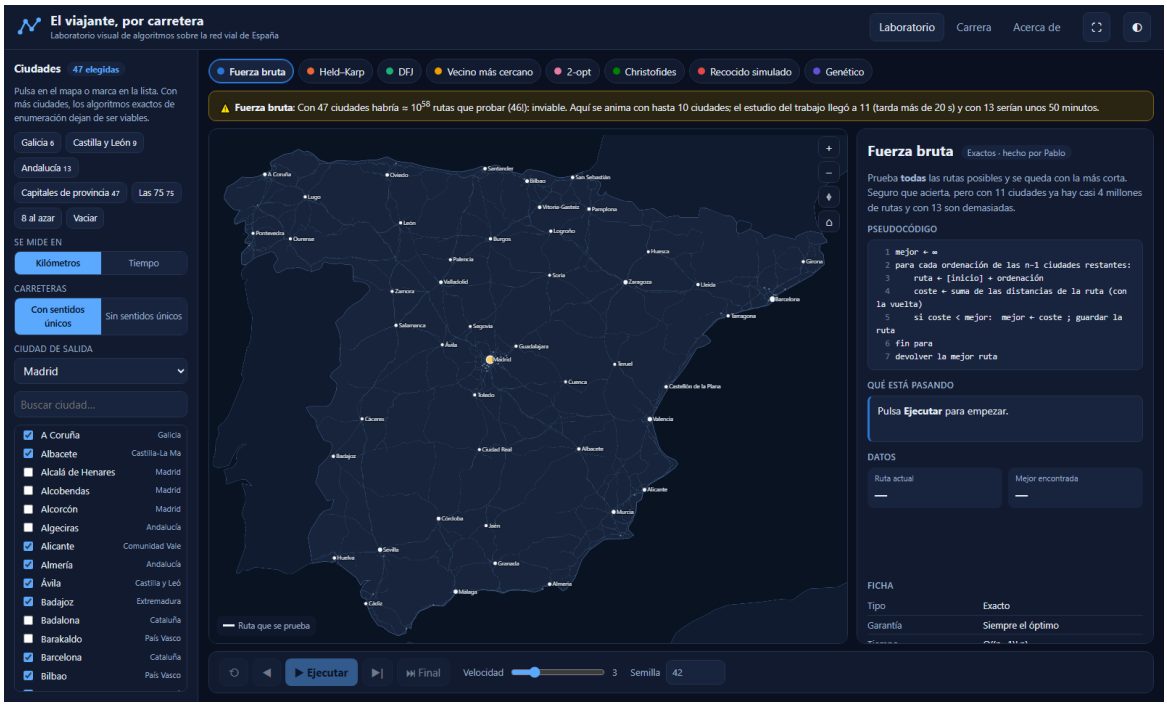


Figura 3.3. Aviso de la fuerza bruta con las 47 capitales: explica por qué es inviable.

3.8 La pestaña Carrera

La *Carrera* pone a competir a varios algoritmos sobre **las mismas ciudades a la vez**. Se usan las ciudades y la versión elegidas en el panel izquierdo.

1. Marca los algoritmos que quieres (por defecto, todos).
2. Pulsa **¡Salida!**. Cada algoritmo avanza durante el mismo tiempo de cálculo, así que el que termina antes es el más rápido de verdad.
3. Al terminar aparece una tabla con el coste, la desviación frente al mejor y el tiempo. un trofeo marca el mejor coste y un rayo el más rápido.

Los algoritmos que no se pueden aplicar se muestran en gris con su motivo.



Figura 3.4. Carrera con las 13 ciudades de Andalucía.

3.9 Guion sugerido para la presentación

Un recorrido de unos 6–8 minutos que cuenta la historia del trabajo (conviene pulsar el botón de pantalla completa y subir un poco el zoom del navegador):

1. **El problema (1 min).** *Galicia*, vecino más cercano, velocidad 3. «Un viajante, seis ciudades: ¿qué ruta?». Mostrar que la **ruta en rojo/amarillo sigue las carreteras reales**.
2. **Fuerza bruta (1 min).** *8 al azar* y *Fuerza bruta*: ver cómo prueba rutas y el contador. Luego *Capitales de provincia*: el aviso muestra por qué es inviable (10^{58} rutas).
3. **Mejorar sin garantías (1,5 min).** *Capitales de provincia*: vecino más cercano (6.772 km) y después 2-opt, viendo cómo quita los cruces (6.326 km).
4. **El óptimo (1,5 min).** *DFJ* con las 47 capitales: los circuitos sueltos de colores, la regla que se añade y la ruta final (**5.917,6 km, el óptimo**).
5. **Inspiración natural (1 min).** *Genético* (rutas al azar que mejoran) o *Recocido*.
6. **La comparación (1,5 min).** Pestaña *Carrera* con *Andalucía*: Held–Karp, DFJ y recocido empatan en el óptimo; el vecino más cercano se queda un 45 % por encima.
7. **Conclusión.** Volver a las diapositivas: complejidad en notación O y qué algoritmo usar en cada caso.

Consejo. Antes de presentar, abre la web una vez y ejecuta DFJ: la primera vez carga el *solver* (2,6 MB) y tarda un segundo más. Con el tema claro se ve mejor en una sala muy iluminada.

3.10 Publicarla en internet (GitHub Pages)

La web **ya está publicada** en `https://raigarper.github.io/MATI_TRABAJO/` gracias a una acción de GitHub (*Publicar web*) que despliega la carpeta `web/` cada vez que cambia algo en ella y llega a la rama `main`. No hay que hacer nada: basta con hacer *push*. Si hace falta lanzarla a mano, está en la pestaña *Actions* del repositorio (*Publicar web* y *Run workflow*). Para que funcione, en *Settings* y *Pages* el origen (*Source*) debe ser **GitHub Actions**, que es como está ahora.

Capítulo 4

Trabajar con los algoritmos

4.1 Cómo funciona cada notebook de algoritmo

Cada algoritmo vive en su propio *notebook* dentro de `parte3/`, con el nombre `algoritmo_autor.ipynb` (por ejemplo, `held_karp_raimundo.ipynb`). Todos tienen la misma estructura: título, **ficha**, **teoría**, **código**, **pruebas** y **resultados**. Una herramienta los integra después en el *notebook* principal, de modo que nadie toca `trabajo_tsp.ipynb` y no hay conflictos de git.

El código de cada algoritmo cumple siempre el mismo **contrato**:

```
def mi_algoritmo(K, inicio=None):
    ...
    return ruta, coste
```

- `K` es el grafo de ciudades: nodos = ciudades; `K[a][b]["peso"]` es la distancia de `a` a `b` (en km u horas, según la versión). Es completo.
- `ruta` es la lista de *todas* las ciudades, una vez cada una, en orden de visita. `coste` es el coste del ciclo **incluida la vuelta**.
- Debe funcionar con `K` dirigido (`nx.DiGraph`) **y** no dirigido (`nx.Graph`); si necesita simetría (como Christofides), debe avisar con un error claro.

Ojo. Dos fallos reales se descubrieron por no probar los dos tipos de grafo: el recocido simulado entraba en un bucle infinito con grafos no dirigidos y DFJ planteaba mal el modelo. Prueba siempre tu algoritmo con `general="grafo_dirigido"` y con `general="grafo_simple_no_dirigido"`.

4.2 Pasos para añadir o cambiar un algoritmo

```
# 1. Ponte en tu rama y trae lo ultimo
git checkout <tu-rama>
git pull origin main

# 2. Copia la plantilla con el nombre de tu algoritmo
cp parte3/_plantilla_algoritmo.ipynb parte3/<algoritmo>_<tunombre>.ipynb

# 3. Trabaja en el notebook, SOLO debajo de la linea con las tijeras
# 4. Anade tu entrada en parte3/fichas.json (copia una existente) y ejecuta:
python parte3/herramientas.py unificar

# 5. Sube tu rama
git add parte3/<algoritmo>_<tunombre>.ipynb parte3/fichas.json
git commit -m "<Algoritmo> (<tunombre>)"
git push origin <tu-rama>
```

Después, en GitHub, abre un *Pull Request* de tu rama a `main`. Quien coordina lo revisa y lo acepta.

4.3 Reglas

- Trabaja **siempre en tu rama**, nunca directamente en `main`, y no renombres las ramas.
- Un *notebook* por algoritmo, con nombre `algoritmo_autor.ipynb` en minúsculas.
- No edites `trabajo_tsp.ipynb`, `data/`, `anexos/` ni la plantilla (así no hay conflictos). Si ves un error ahí, avisa.
- Dale a cada función un **nombre único** (`fuerza_bruta`, `held_karp`…): cuando se integren todos en un solo *notebook*, los nombres no deben chocar.
- No crees archivos `.py` propios con el algoritmo: el código va en el *notebook*. Usa solo librerías de `requirements.txt`.
- **Si usas IA, apunta qué le has pedido** en tu ficha y en `REGISTRO.md`.
- Si git avisa de un **conflicto** o algo no cuadra, no lo fuerces: avisa.

4.4 Integrar los algoritmos (quien coordina)

```
python parte3/herramientas.py unificar    # fichas, kernel y formato estandar
python parte3/herramientas.py integrar    # copia lo que hay bajo las tijeras a la Parte
     3 del notebook
```

Después se abre `trabajo_tsp.ipynb` y se ejecuta entero (*Restart & Run All*). La herramienta es repetible: cada vez *sustituye* lo integrado antes y avisa si dos *notebooks* definen una función con el mismo nombre.

Si cambia el código de la Parte 2 del *notebook* principal, hay que regenerar la plantilla: `python parte3/herramientas.py plantilla`.

4.5 Pedir una versión del grafo

```
K = cargar_version(aristas="completo", nodos="capitales",
                   ambito="espana", peso="km", general="grafo_dirigido")
```

Tabla 4.1. Parámetros de `cargar_version`.

Parámetro	Valores
`aristas`	`completo`, `autovias_principales`, `solo_autovias`, `sin_autovias`
`nodos`	`capitales` (47) o `capitales_y_100k` (75)
`ambito`	`espana` o una comunidad: `galicia`, `andalucia`, `castilla_y_leon`, `cataluna`…
`peso`	`km` o `tiempo` (en segundos)
`general`	`grafo_dirigido` o `grafo_simple_no_dirigido` (los `multigrafo_*` dan lo mismo)

Para probar rápido, usa una comunidad (`galicia`: 6 ciudades; `andalucia`: 13). Para un algoritmo exacto de enumeración con pocas ciudades, coge un trozo de `K` con `K.subgraph(lista_de_ciudades)`. Todas las versiones están en `data/versiones/catalogo.csv`.

4.6 Qué escribir en tu *notebook*

Debajo de la línea de las tijeras, en este orden (borra las notas en cursiva de la plantilla cuando rellenes cada parte):

1. **Teoría:** cómo funciona (en lenguaje llano y con precisión), pseudocódigo, **complejidad en tiempo y memoria** y propiedades (¿exacto?, ¿determinista?, ¿garantía?, ¿necesita simetría?, ¿hasta cuántas ciudades?).
2. **Código:** tu función, con un comentario por bloque lógico. Si usas azar, fija una semilla.
3. **Pruebas:** una prueba rápida con Galicia y las demás comprobaciones, con `validar_ruta`.
4. **Resultados:** una tabla con lo *medido* (versión, ciudades, coste, tiempo, memoria). Que las cifras salgan de la ejecución, no escritas a mano.

Capítulo 5

Notebook, estudio y datos

5.1 Las cinco partes del *notebook*

Tabla 5.1. Partes de `trabajo_tsp.ipynb`.

Parte	Qué es	Necesita
1	Construcción del grafo (carga y dibujo)	El grafo grande (se puede saltar).
2	Versiones del grafo y `cargar_version`	Solo `data/versiones/`.
3	Los ocho algoritmos, con pruebas y resultados	Nada más.
4	Estudio empírico (análisis y figuras)	Los CSV de `data/resultados/`.
5	Conclusiones y complejidad en notación O	Nada.

5.2 Repetir el estudio empírico

El estudio ejecuta los ocho algoritmos sobre las 240 versiones del grafo y sobre subconjuntos de 5 a 75 ciudades. Se hace en **dos pasadas**, porque el coste de una ruta no depende de la carga del equipo pero el tiempo sí:

```
python anexos/ejecutar_estudio.py           # en paralelo, unos 42 min (costes y
    calidad)
python anexos/ejecutar_estudio.py --serie   # en serie, unos 15 min (tiempos y memoria)
python anexos/ejecutar_estudio.py --prueba  # version reducida, unos minutos
```

- Cada resultado se guarda al llegar: si se interrumpe, **al volver a lanzarlo reanuda** donde se quedó. `--forzar` lo repite todo desde cero.
- Si pasan 15 minutos sin ningún resultado nuevo, se detiene solo (algo se ha atascado).
- Durante la pasada **en serie**, deja el equipo en reposo y enchufado: cualquier otro programa falsea los tiempos.

Para que el *notebook* vuelva a calcularlo todo, cambia `RECALCULAR_ESTUDIO = True` en la Parte 4. Los resultados quedan en `data/resultados/` (`estudio_resultados.csv`, `tiempos_serie.csv`, `no_aplicables.csv` y `calibracion.csv`).

5.3 Regenerar el grafo grande

Solo hace falta si no tienes el archivo `.graphml`:

```
python anexos/descargar_red.py       # ~1,4 GB, ~7 min, reanudable
python anexos/construir_grafo.py     # ~4 min, ~2,3 GB de RAM
```

Si el grafo ya existe, ambos avisan y no hacen nada (`--forzar` para repetir). La web no necesita este grafo, pero si hubiera que regenerar sus datos (`web/data/`) se usa `python web/herramientas/preparar_datos.py`, que sí lo necesita (unos 4 GB de memoria libre).

Capítulo 6

Compilar la memoria y esta guía

La memoria y esta guía están escritas en LaTeX y se convierten en PDF con **Tectonic**, un compilador de un solo ejecutable que no necesita instalación y descarga los paquetes que le hacen falta.

6.1 Obtener Tectonic

Descarga el archivo para Windows (`x86_64-pc-windows-msvc`) desde la página de versiones del proyecto (`github.com/tectonic-typesetting/tectonic/releases`), descomprímelo y deja `tectonic.exe` en una carpeta de tu elección (o en el `PATH`). Se comprueba con `tectonic --version`. En otros sistemas se puede instalar con el gestor de paquetes habitual.

6.2 Compilar

```
# La memoria
cd memoria
tectonic -X compile memoria.tex          # genera memoria.pdf

# Esta guia
cd ../guia
tectonic -X compile guia_de_uso.tex      # genera guia_de_uso.pdf
```

La primera vez descarga paquetes y tarda un par de minutos; después es rápido. Si aparecen avisos de tipo *Overfull \hbox*, significa que alguna línea se sale del margen: conviene corregirlos.

6.3 Regenerar las figuras de la memoria

Las gráficas y mapas de la memoria salen de los resultados del estudio:

```
python memoria/figuras/generar_figuras.py
```

Genera los PDF de `memoria/figuras/`. Si se repite el estudio, hay que regenerarlas y volver a compilar. Las capturas de pantalla de la web están en `web/capturas/`.

6.4 Convenciones tipográficas

El documento usa la fuente *Libertinus* (texto y fórmulas) a 11 pt, interlineado holgado, números con coma decimal y fórmulas numeradas. Los capítulos empiezan en página nueva, y los títulos de apartado no se quedan solos al pie de página. Si añades contenido, respeta estas reglas: tablas con `tabularx` (que no se salgan del margen), figuras con `[htbp]` y fórmulas cortas.

Capítulo 7

Problemas frecuentes

Tabla 7.1. Problemas habituales y qué hacer.

Problema	Qué hacer
La web sale en blanco o dice «No se han podido cargar los datos»	Se ha abierto `index.html` haciendo doble clic. Usa `iniciar_web.bat` o `python -m http.server` desde `web/`.
`python -m http.server` dice que el puerto está ocupado	Usa otro puerto: `python -m http.server 8080` y abre `http://localhost:8080`.
DFJ tarda en empezar la primera vez en la web	Es normal: carga el *solver* (2,6 MB). Después es inmediato.
La fuerza bruta o Held–Karp no se ejecutan	El aviso amarillo explica el límite: reduce el número de ciudades.
Christofides no se ejecuta	Elige «Sin sentidos únicos» (necesita distancias simétricas).
El mapa se ve pequeño o desplazado	Pulsa ⊕ (encuadrar las ciudades) o △ (toda la península).
Error al importar `pulp` o el *solver* CBC	`pip install -r requirements.txt` (instala `pulp[cbc]`).
El *notebook* se detiene o el equipo va lento	Falta memoria: cierra el navegador y otros programas; la Parte 1 necesita varios gigabytes. Salta la Parte 1 si no necesitas el grafo grande.
Un algoritmo funciona con grafo dirigido pero falla con no dirigido	Revisa cómo lee las aristas: en un `nx.Graph` cada arista aparece *una sola vez*. Convierte con `K.to_directed()` o guarda los dos sentidos.
Git dice que hay un conflicto	No lo fuerces: avisa antes de continuar.
GitHub envía un correo de «Publicar web: all jobs have failed»	Mira el detalle en la pestaña *Actions*. Si dice que Pages no está habilitado, el propietario debe revisar *Settings*, *Pages* y comprobar que el origen es GitHub Actions.
Aparecen avisos *Overfull \hbox* al compilar	Una línea o tabla se sale del margen: acórtala, usa `tabularx` o permite salto de línea.

7.1 Glosario

Grafo dirigido / no dirigido.
Con sentidos únicos, ir de A a B puede costar distinto que de B a A; sin ellos, cuesta lo mismo.

Exacto.
Algoritmo que siempre encuentra la mejor ruta.

Heurística.
Método rápido que encuentra una buena ruta, pero sin garantía de que sea la mejor.

Metaheurística.
Heurística que usa el azar para explorar muchas rutas (recocido simulado, genético).

Desviación.
Cuánto por encima del óptimo queda una ruta: $(C - C^*)/C^*$, en porcentaje.

Notación O.
Cómo crece el tiempo (o la memoria) al aumentar el número de ciudades n: $O(n^2)$ se multiplica por 4 al duplicar n; $O(2^n)$ se duplica con cada ciudad nueva.

Óptimo local.
Ruta que no se puede mejorar con pequeños cambios, aunque no sea la mejor posible.

Subciclo (circuito suelto).
En DFJ, un ciclo que no pasa por todas las ciudades y que hay que eliminar con reglas adicionales.